



This assignment has been designed to help students build a database with multiple tables, focusing on the relationships between these tables and cleaning up poorly formatted data.

COMP 2003 Relational databases

ASSIGNMENT 3

Group – 10%

Assignment 3 – Relational Databases

This assignment requires you to apply your understanding of table relationships including Primary and Foreign Keys and Cardinality (i.e. relationships between tables, such as one-to-one, many-to-one and one-to-many). You will also combine your knowledge of SELECT, UPDATE, and ALTER TABLE statements, JOIN and UNION clauses, and REPLACE, TRIM and SUBSTRING_INDEX string functions

to build carefully constructed statements that will clean and format heterogeneous/irregular data. Your mark in this assignment counts for 10% of your final grade.

Assignment Requirements:

This assignment is to be completed in groups of 1-5 people.

SQL formatting

- Please include your full names and student numbers
- Name your SQL script *GroupNumber_Assignment3.sql* and complete all of your SQL work in this file. Ensure you use the data from the *data.zip* file to populate your database.
- Inside *GroupNumber_Assignment3.sql* I should find all your SQL commands required to complete the assignment.
- The data provided in *data.zip* is solely for the purpose of populating your database with data – do not complete your SQL work in this file.
- Utilize the SQL Standards document provided in week 2 – with note to capitalization of keywords and lower-case database, table and field names.
- Statements should be broken over multiple lines as appropriate
- Remember to add a brief descriptive comment for each query. You do not need to document every single line, but please annotate the purpose or function of the statement. You must demonstrate that you understand what the SQL statement is doing.
- Please write complete sentences in the commenting for relevant questions.

Report formatting

- Name your report file *GroupNumber_Assignment3.docx* • Give your report an appropriate title with a separate title page.
- Please include each student's full name and student number.
- For each question, please include the questions' number and text.
- When providing documentation of a SQL query, please include the text of your query, including comments, and a screenshot.
- Screenshots should include the script statement being executed, the Schemas tab, the Output pane, and the Result Grid if applicable.

Submitting your assignment

- Create a zip file containing your SQL script and report
- Submit assignment zip file via the course website. Do not submit via email.

Evaluation Method

For each question you will receive the following marks:

- Execution: A variable number of marks will be assigned whether the command will run as provided.
- Accuracy: A variable number of marks will be assigned whether the question is answered correctly.

Additionally, the following marks will be assigned to the SQL script as a whole

- Structure: Up to **[16 marks]** will be assigned for well-structured scripts that follow SQL standards. i.e. capitalized commands, lower case field names with underscores where needed, new lines, etc.
- Documentation: Up to **[10 marks]** will be assigned for helpful, descriptive comments and well-structured screenshots to be included throughout the script and report.

If you have any questions, please do not hesitate to ask me.

Good luck.

Part One: Students Table

Step 1 – Create a database that will be used for this assignment. Name the database appropriately and instruct MySQL to make that database default for the remainder of the assignment.

Execution: 2 marks

Step 2 – Use the statement provided below to build a table to hold information about our class.

```
CREATE TABLE `students` (  
  `ID` int NOT NULL AUTO_INCREMENT,  
  `first_name` varchar(45) DEFAULT NULL,  
  `last_name` varchar(45) DEFAULT NULL,  
  `attended` INT DEFAULT NULL,  
  PRIMARY KEY (`ID`)  
) ENGINE=InnoDB;
```

Step 3 – Use Table Data Import Wizard to add data from a CSV file to the students table you created in the last step. HINT Make sure the Source Column matches with your desired Destination Column.

Provide a screenshot of your *Configure Import Settings* window and document this step in the comments of your assignment script.

Documentation: 1 mark

Part Two: Teams Table

Step 4 – Use Table Data Import Wizard to add data from a CSV file to a new table called *team_members*.

HINT Make sure that the Field Type that has been assigned by MySQL is the most appropriate choice.

Provide a screenshot of your *Configure Import Settings* window and document this step in the comments of your assignment script.

Documentation: 1 mark

Step 5 – The current *team_members* table is not suitable for long term storage because it violates 1NF. Write an SQL statement that creates a table named *teams_temp* with two columns:

member – variable length text with max length of 45 characters

team – variable length text with max length of 45 characters

Execution: 1 mark

Accuracy: 1 mark

Step 6 – Take a moment to view the data in the *team_members* table. Write an SQL query that populates the *teams_temp* table by selecting data from the *team_members* table. Use an INSERT statement to combine the *Member 1*, *Member 2*, *Member 3*, *Member 4*, and *Member 5* columns along with their corresponding Team Name values using the UNION operator.

Execution: 1 mark

Accuracy: 5 marks

Step 7 – Write an SQL query that deletes rows from the *teams_temp* table where the member column is empty.

Execution: 1 mark

Accuracy: 1 mark

Step 8 – Write an SQL query that updates the *teams_temp* table by performing the following actions:

Remove the prefix 'Team ' from the *team* column.

Remove the prefix 'Team:- ' from the *team* column.

Trim any leading or trailing spaces from the *team* column.

Execution: 1 mark

Accuracy: 4 marks

Step 9 – Write an SQL command that drops the table named *team_members*.

Execution: 1 mark

Step 10 There are at least two records that contain duplicate records. Write an SQL query that deletes rows from the *teams_temp* table where the member column is either 'Anthony' or 'Feinit'. In your report, state whether there are additional duplicates, and how you searched for them.

Execution: 1 mark

Written Answer: 2 marks

Accuracy: 3 marks

Step 11 – Write an SQL query that creates a new table named *teams* with two columns: id (integer), which serves as the primary key, and *team_name* (variable length text with max 45 characters). Populate the *team_name* column with distinct values from the *teams_temp* table, ordered alphabetically.

Execution: 1 mark

Accuracy: 4 marks

Step 12 – Write an SQL statement that adds a new column named *team_name* to the existing *students* table. The *team_name* column should have a data type of VARCHAR(45).

Execution: 1 mark

Accuracy: 1 marks

Step 13 – Write an SQL statement that accomplishes the following:

Update the *students* table: For students who attended the event (where *attended* = 1), set their *team_name* based on the *team* value in the *teams_temp* table. Match students' first names with the corresponding team members in *teams_temp*.

Use an INNER JOIN to connect the *students* and *teams_temp* tables based on the first name.

Update the *students* table by setting the *team_name* column to the corresponding team name from *teams_temp*.

Use the INNER JOIN clause to combine the relevant rows from both tables.

Use the SUBSTRING_INDEX function to extract the first name from the *teams_temp.member* column.

Set the *students.team_name* to the corresponding *teams_temp.team* value for students who attended the event.

Execution: 1 mark

Accuracy: 7 marks

Step 14 – Alter the *students* table to add a new integer column named *team_ID*.

Execution: 1 mark

Accuracy: 1 mark

–
Step 15 Write an SQL statement that accomplishes the following:

Update the students table: For students who attended an event (where *attended* = 1), set their *team_id* based on the corresponding *team name* from the *teams* table.

Join the tables: Use an INNER JOIN to connect the *students* and *teams* tables based on the *team name*.

Set the team ID: Update the *students* table by assigning the correct *team_id* from the *teams* table.

Execution: 1 mark

Accuracy: 5 marks

Step 16 – Write an SQL statement that drops the column called *team_name* from the *students* table.

Execution: 1 mark

Step 17 – We are now done with the *teams_temp* table. Drop it from our database.

Execution: 1 mark

Part Three: Pilots Table

Step 18 – Go into Excel and save the *pilots.xlsx* as a .csv file, then use Table Data Import Wizard to add data from a CSV file to a new table called *pilots*. Don't forget to make sure that the Field Type that has been assigned by MySQL is the most appropriate choice. Provide a screenshot of your *Configure Import Settings* window and document this step in the comments of your assignment script.

Documentation: 1 mark

Step 19 – Write an SQL query that accomplishes the following:

Rename the columns:

Change the column name Team number to *team_id*.

Change the column name Height (in m) to *height_m*.

Change the column name Arm length (in m) to *arm_length_m*.

Change the column name Handedness (right|left) to *handedness*.

Execution: 1 mark

Accuracy: 3 marks

Step 20 Update the *pilots* table to remove the prefix 'Team ' from the *team_id* values.

Execution: 1 mark

Accuracy: 3 marks

–
Step 21 – Update the *pilots* table to set *team_id* to *NULL* where the current value is an empty string.

Execution: 1 mark

Accuracy: 3 marks

Step 22 – In the *pilots* table there are height measurements. However, there is some inconsistency in the height units recorded. Some heights are in meters (followed by the letter 'm'), while others are in centimeters and also have the letter 'c'. Your task is to write an SQL query that accomplishes the following:

Update the *height_m* column:

Remove any occurrences of the letter 'm' from the height values.

Remove any occurrences of the letter 'c' from the height values.

Execution: 1 mark

Accuracy: 4 marks

Step 23 – Since some of the height values were measured in centimeters, let's calculate the proper meter value.

Update the *height_m* column:

Divide the current value by 100 when the current value is greater than 100.

Execution: 1 mark

Accuracy: 2 marks

Step 24 – The gender column contains values that are not standardized.

Update the *gender* column:

Where the current value starts with 'm' set the value to 'M'

Where the current value starts with 'f' set the value to 'F'

Otherwise set the value to NULL

Execution: 1 mark

Accuracy: 3 marks

Step 25 In the *arm_length_m* column there is some inconsistency in the units recorded. Some lengths are in meters (followed by the letter 'm'), while others are in centimeters and also have the letter 'c'. Similar to step 23:

Update the *arm_length_m* column:

–
Remove any occurrences of the letter 'm' from the height values.

Remove any occurrences of the letter 'c' from the height values.

Execution: 1 mark

Accuracy: 4 marks

Step 26 – Since some of the length values were measured in centimeters, let's calculate the proper meter value.

Update the *arm_length_m* column:

Divide the current value by 100 when the current value is greater than 3.

Execution: 1 mark

Accuracy: 2 marks

Step 27 – The *handedness* column contains values that are not standardized.

Update the *handedness* column:

Where the current value starts with 'R' set the value to 'R'

Where the current value starts with 'L' set the value to 'L'

Otherwise set the value to NULL

Execution: 1 mark

Accuracy: 4 marks

Step 28 – We need to be able to link *pilots* with the *students* table. Add an integer column named *student_id* to the *pilots* table

Execution: 1 mark

Step 29 We need to assign *student_id* in the *pilots* table to match the *id* value from the *students* table.

Write an SQL query that accomplishes the following:

Update the pilots table:

Use an INNER JOIN to connect the pilots and students tables based on the first name.

Set the pilots.student_id to the corresponding students.id.

Hint: you can isolate the first name from the *name* column using a substring index function.

Execution: 1 mark

Accuracy: 4 marks

–
Step 30 – There is a duplicate record. Delete pilot 3 from the *pilots* table (ID = 3).

Execution: 1 mark

Step 31 – The *team_id* and *height_m* values got swapped for pilot 4 (ID = 4). Set their *team_id* to 4 and their *height_m* to NULL.

Execution: 1 mark

Accuracy: 2 marks

Step 32 – Once data is clean, we can alter column data types to desirable configurations. Modify specific columns in the table to ensure the following requirements:

Team_id – integer

Height_m – float, capable of handling values up to 9.99, two decimal places

Gender – fixed length text of 1 character

Arm_length_m - float, capable of handling values up to 9.99, two decimal places

Handedness – fixed length text of 1 character

Drop the *name* column

Execution: 1 mark

Accuracy: 3 marks

Part Four: Airplanes Table

Along with information contained in a CSV, we are going to be uploading images into a specially formatted column for storage within our database.

Step 33 Use Table Data Import Wizard to add data from a CSV file to a new table called *airplanes*. Make sure that the Field Type that has been assigned by MySQL is the most appropriate choice. Provide a screenshot of your *Configure Import Settings* window and document this step in the comments of your assignment script.

Documentation: 1 mark

Step 34 – Alter the *airplanes* table to have a new column called *image* and ensure that the data type is LONGBLOB

Execution: 1 mark

Step 35 – MySQL will only allow you to upload images from specific directories and this is set during the installation of MySQL. Run the following statement to find out what directory you will need to save your images to.

—
`SHOW VARIABLES LIKE 'secure_file_priv';`

You will likely receive a value similar to this: `C:\ProgramData\MySQL\MySQL Server 8.0\Uploads\`

Ensure that you copy the images provided for this assignment INTO THAT DIRECTORY.

Execution: 1 mark

Step 36 – Use this statement as a template. It will load the *Plane1.png* image file from the directory listed into the corresponding plane record (*WHERE id = 1*) in the *airplanes* table. **Note that the filepath has forward slashes.**

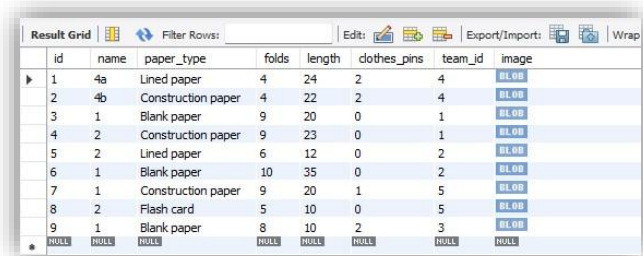
```
UPDATE airplanes
```

```
SET image = LOAD_FILE('C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Plane1.png')
```

```
WHERE id = 1;
```

Execution: 1 mark

Step 37 – Using Step 36 as a template, write statements to load images for each of the 9 planes. Pay attention to the file extension (some are .png files, others are .jpg files). When you are successful you'll see a blob has been loaded into the *image* column – see image below.



	id	name	paper_type	folds	length	clothes_pins	team_id	image
▶	1	4a	Lined paper	4	24	2	4	BLOB
	2	4b	Construction paper	4	22	2	4	BLOB
	3	1	Blank paper	9	20	0	1	BLOB
	4	2	Construction paper	9	23	0	1	BLOB
	5	2	Lined paper	6	12	0	2	BLOB
	6	1	Blank paper	10	35	0	2	BLOB
	7	1	Construction paper	9	20	1	5	BLOB
	8	2	Flash card	5	10	0	5	BLOB
*	9	1	Blank paper	8	10	2	3	BLOB
		NULL	NULL	NULL	NULL	NULL	NULL	NULL

Execution: 5 marks

Documentation: 1 mark

Part Five: Linking Tables

Step 38 – Let's put the finishing touches on our tables to ensure that we have the proper relationships established in our database. Add Primary and Foreign Key constraints to the database according to the Entity Relationship Diagram below. Yellow key icons are primary keys. Red diamonds are foreign keys.

Execution 7 marks

